Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа №2

по курсу «Численные методы»

«Сравнительный анализ методов численного решения краевой задачи для линейного дифференциального уравнения второго порядка»

Студент группы ИУ9-62Б: Трофименко Д. И.

Преподаватель: Домрачева А. Б.

Москва 2026

1 Постановка задачи

1.1 Дано:

Краевая задача для линейного дифференциального уравнения второго порядка:

$$y'' + p(x)y' + q(x)y = f(x)$$

$$y(0) = a$$

$$y(1) = b$$

1.2 Найти:

- Аналитическое решение задачи Коши:

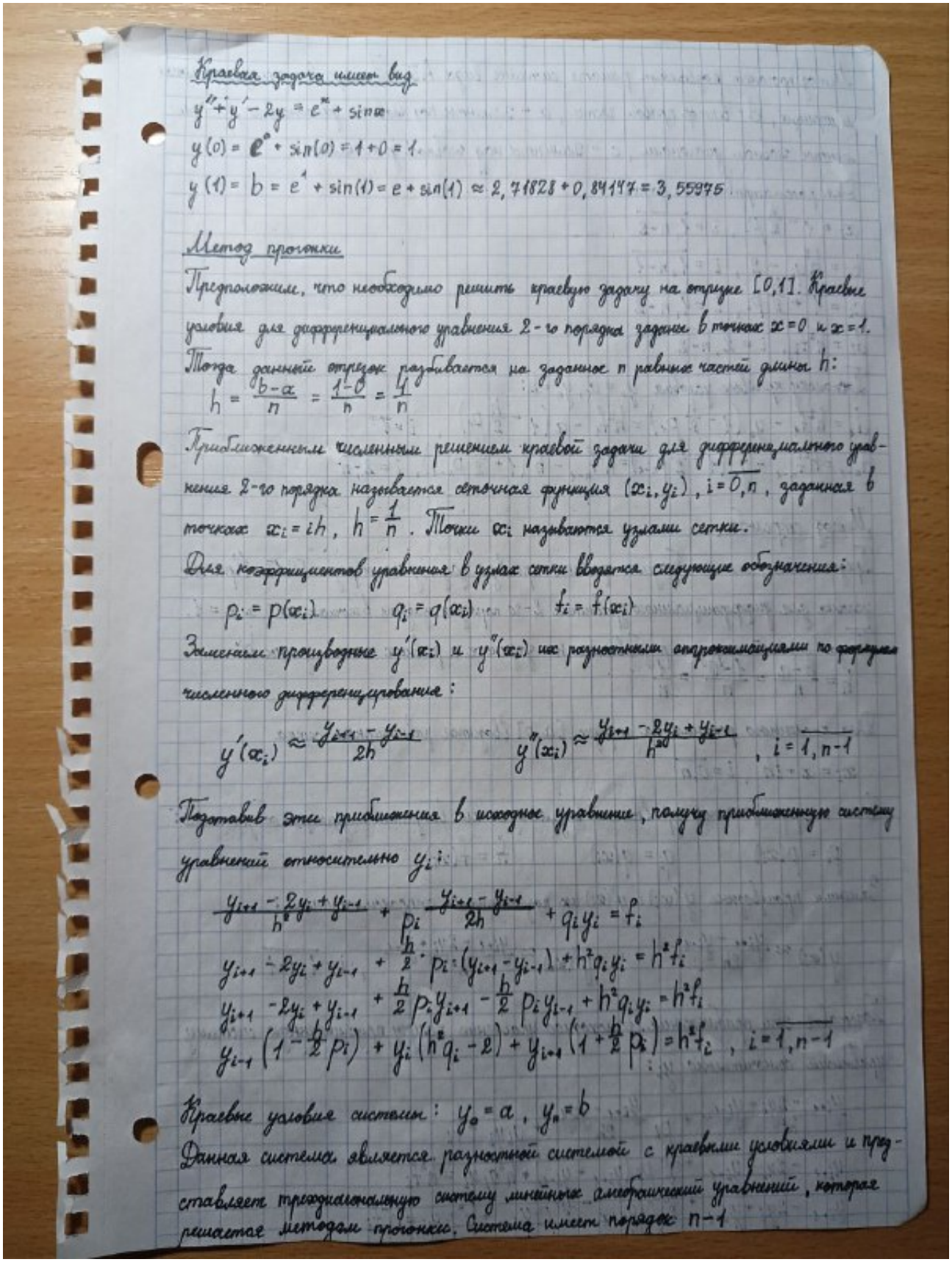
$$y'' + p(x)y' + q(x)y = f(x),\ y(0) = y_0,\ y'(0) = y'_0;$$

- По найденному решению задачи Коши вычислить $b = y(1)$;
- С помощью метода прогонки и метода стрельбы найти численное решение $(x_i, y_i),\ i = \overline{0, n}$ краевой задачи для того же уравнения с краевыми условиями $y(0) = a,\ y(1) = b$;
- Вычислить $|y_i - y_i^*|,\ i = \overline{0, n}$, найти погрешность численного решения $\|y - y^*\| = \max\limits_{0 \le i \le n} |y_i - y_i^*|$ (y – аналитическое решение, y^* – численное решение) и сравнить метод прогонки и метод стрельбы.

1.3 Индивидуальный вариант:

В качестве индивидуального варианта были взяты следующие значения:
$p(x) = 1$
$q(x) = -2$
$f(x) = cos(x) - 3sin(x)$
$y_0 = y(0) = 1$
$y'_0 = y'(0) = 2$

2 Теоретические сведения



Краевая задача имеет вид

$$y'' + y' - 2y = e^x + \sin x$$

$$y(0) = e^0 + \sin(0) = 1 + 0 = 1$$

$$y(1) = b = e^1 + \sin(1) = e + \sin(1) \approx 2,71828 + 0,84147 = 3,55975$$

Метод прогонки

Предположим, что необходимо решить краевую задачу на отрезке $[0,1]$. Краевые условия для дифференциального уравнения 2-го порядка заданы в точках $x=0$ и $x=1$. Тогда данный отрезок разбивается на заданное n равных частей длины h:

$$h = \frac{b-a}{n} = \frac{1-0}{n} = \frac{1}{n}$$

Приближенным численным решением краевой задачи для дифференциального уравнения 2-го порядка называется сеточная функция (x_i, y_i), $i = \overline{0,n}$, заданная в точках $x_i = ih$, $h = \frac{1}{n}$. Точки x_i называются узлами сетки.

Для коэффициентов уравнения в узлах сетки вводятся следующие обозначения:

$$p_i = p(x_i) \qquad q_i = q(x_i) \qquad f_i = f(x_i)$$

Заменим производные $y'(x_i)$ и $y''(x_i)$ их разностными аппроксимациями по формуле численного дифференцирования:

$$y'(x_i) \approx \frac{y_{i+1} - y_{i-1}}{2h} \qquad y''(x_i) \approx \frac{y_{i+1} - 2y_i + y_{i-1}}{h^2}, \quad i = \overline{1, n-1}$$

Подставив эти приближения в исходное уравнение, получу приближенную систему уравнений относительно y_i:

$$\frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} + p_i \frac{y_{i+1} - y_{i-1}}{2h} + q_i y_i = f_i$$

$$y_{i+1} - 2y_i + y_{i-1} + \frac{h}{2} \cdot p_i (y_{i+1} - y_{i-1}) + h^2 q_i y_i = h^2 f_i$$

$$y_{i+1} - 2y_i + y_{i-1} + \frac{h}{2} p_i y_{i+1} - \frac{h}{2} p_i y_{i-1} + h^2 q_i y_i = h^2 f_i$$

$$y_{i-1}\left(1 - \frac{h}{2} p_i\right) + y_i\left(h^2 q_i - 2\right) + y_{i+1}\left(1 + \frac{h}{2} p_i\right) = h^2 f_i, \quad i = \overline{1, n-1}$$

Краевые условия системы: $y_0 = a$, $y_n = b$

Данная система является разностной системой с краевыми условиями и представляет трехдиагональную систему линейных алгебраических уравнений, которая решается методом прогонки. Система имеет порядок $n-1$

Рис. 1 — Теоретические сведения (1 часть)

Метод прогонки позволяет решать системы вида $A\bar{x} = \bar{d}$, где A – трехдиагональная матрица, d – вектор правой части, a – элементы под главной диагональю, b – элементы главной диагонали, c – элементы над главной диагональю

Для рассматриваемой задачи:

$$a_i = 1 - \frac{h}{2} p_i, \quad i = \overline{1, n-2}$$

$$b_i = h^2 q_i - 2, \quad i = \overline{1, n-1}$$

$$c_i = 1 + \frac{h}{2} p_i, \quad i = \overline{1, n-2}$$

$$d_i = h^2 f_i, \quad i = \overline{2, n-2}$$

Поскольку краевые условия $y_0 = a$, $y_n = b$:

$$d_1 = h^2 f_1 - y_0 \left(1 - \frac{h}{2} p_1\right) = h^2 f_1 - a\left(1 - \frac{h}{2} p_1\right), \quad i = 1$$

$$d_{n-1} = h^2 f_{n-1} - y_n \left(1 + \frac{h}{2} p_{n-1}\right) = h^2 f_{n-1} - b\left(1 + \frac{h}{2} p_{n-1}\right), \quad i = n-1$$

<u>Метод стрельбы</u>

Предположим, что необходимо решить краевую задачу на отрезке $[0,1]$. Краевые условия для дифференциального уравнения 2-го порядка заданы в точках $x = 0$ и $x = 1$. Тогда данный отрезок разбивается на заданное n равных частей длины h:

$$h = \frac{b-a}{n} = \frac{1-0}{n} = \frac{1}{n}$$

Для численного решения на отрезке $[a, b]$ вводится равномерная сетка

$$x_i = a + ih, \quad i = \overline{0, n}$$

Для коэффициентов уравнения в узлах сетки вводятся следующие обозначения:

$$p_i = p(x_i) \qquad q_i = q(x_i) \qquad f_i = f(x_i)$$

Заменим производные $y'(x_i)$ и $y''(x_i)$ их разностными аппроксимациями:

$$y'(x_i) \approx \frac{y_{i+1} - y_{i-1}}{2h} \qquad y''(x_i) \approx \frac{y_{i+1} - 2y_i + y_{i-1}}{h^2}, \quad i = \overline{1, n-1}$$

Подставив эти приближения в исходное уравнение, получу приближенную систему уравнений относительно y_i:

$$\frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} + p_i \frac{y_{i+1} - y_{i-1}}{2h} + q_i y_i = f_i$$

$$y_{i+1} - 2y_i + y_{i-1} + \frac{h}{2} p_i (y_{i+1} - y_{i-1}) + h^2 q_i y_i = h^2 f_i$$

$$y_{i+1} - 2y_i + y_{i-1} + \frac{h}{2} p_i y_{i+1} - \frac{h}{2} p_i y_{i-1} + h^2 q_i y_i = h^2 f_i$$

Рис. 2 — Теоретические сведения (2 часть)

$$y_{i-1}\left(1-\frac{h}{2}p_i\right)+y_i\left(h^2q_i-2\right)+y_{i+1}\left(1+\frac{h}{2}p_i\right)=h^2f_i\ ,\quad i=\overline{1,n-1}$$

Сеточное решение буду искать в виде:

$$y[i]=y_0[i]+C_1y_1[i]\ ,$$

где $y_0[i]=y_0(x_i)$, $y_1[i]=y_1(x_i)$. Будем искать решения, удовлетворяющие условиям:

$$y_0[0]=A \qquad y_0[1]=D_0 \qquad y_1[0]=0 \qquad y_1[1]=D_1\neq 0\ ,$$

причём для уменьшения вычислительной погрешности $D_0=A+O(h)$, $D_1=O(h)$,

а также $y_0[i]$ – частное решение , $y_1[i]$ – часть общего решения

Для определения y_0 получим уравнение:

$$\frac{y_0[i+1]-2y_0[i]+y_0[i-1]}{h^2}+p_i\frac{y_0[i+1]-y_0[i-1]}{2h}+q_iy_0[i]=f_i$$

$$y_0[i+1]-2y_0[i]+y_0[i-1]+\frac{p_ih}{2}y_0[i+1]-\frac{p_ih}{2}y_0[i-1]+q_ih^2y_0[i]=h^2f_i$$

$$\left(1+\frac{p_ih}{2}\right)y_0[i+1]+\left(q_ih^2-2\right)y_0[i]+\left(1-\frac{p_ih}{2}\right)y_0[i-1]=h^2f_i$$

$$\left(1+\frac{p_ih}{2}\right)y_0[i+1]=h^2f_i+\left(2-q_ih^2\right)y_0[i]-\left(1-\frac{p_ih}{2}\right)y_0[i-1]$$

$$y_0[i+1]=\frac{f_ih^2+\left(2-q_ih^2\right)y_0[i]-\left(1-\frac{p_ih}{2}\right)y_0[i-1]}{1+\frac{p_ih}{2}}\ ,\quad i=\overline{1,n-1}$$

Для определения y_1 получим уравнение:

$$\frac{y_1[i+1]-2y_1[i]+y_1[i-1]}{h^2}+p_i\frac{y_1[i+1]-y_1[i-1]}{2h}+q_iy_1[i]=0$$

...

$$y_1[i+1]=\frac{\left(2-q_ih^2\right)y_1[i]-\left(1-\frac{p_ih}{2}\right)y_1[i-1]}{1+\frac{p_ih}{2}}\ ,\quad i=\overline{1,n-1}$$

После этого последовательно вычисляются значения:

$$y_0[2],\ y_0[3],\ \dots,\ y_0[n]- \qquad y_1[2],\ y_1[3],\ \dots,\ y_1[n]$$

Правое граничное условие $y[n]=B$. Подставляя его в уравнение $y[n]=y_0[n]+C_1y_1[n]$,

получаем $y_0[n]+C_1y_1[n]=B$

Тогда можно найти C_1:

$$C_1=\frac{B-y_0[n]}{y_1[n]}$$

Искомое решение в каждом узле сетки определяется по формуле:

$$y[i]=y_0[i]+C_1y_1[i]\ ,\quad i=\overline{0,n}$$

Рис. 3 — Теоретические сведения (3 часть)

3 Практическая реализация

Исходный код программы представлен в листингах 3– 2.

Листинг 1 — lab2.py. (1 часть)

```
import math

def exact_solution(x):
    return math.exp(x) + math.sin(x)

def p(x):
    return 1.0

def q(x):
    return -2.0

def f(x):
    return math.cos(x) - 3.0 * math.sin(x)

def print_method_table(title, x_values, y_exact, y_num, errors):
    print(title)
    print(f"{'x':<10}{'y':<22}{'y*':<22}{'|y_i - y*_i|':<22}")
    for i in range(len(x_values)):
        print(
            f"{x_values[i]:<10.1f}"
            f"{y_exact[i]:<22.15f}"
            f"{y_num[i]:<22.15f}"
            f"{errors[i]:<22.15f}"
        )
    print()
    print(f"||y - y*|| = max|y_i - y*_i| = {max(errors):.15f}")
    print()

def running_method():
    a = 0.0
    b = 1.0
    n = 10
    h = (b - a) / n
    y_left = 1.0
    y_right = math.exp(1.0) + math.sin(1.0)
    x_values = [a + i * h for i in range(n + 1)]
    alpha = [0.0] * (n + 1)
    beta = [0.0] * (n + 1)
    y_num = [0.0] * (n + 1)
    for i in range(1, n):
        xi = x_values[i]
        A = 1.0 - p(xi) * h / 2.0
        B = q(xi) * h * h - 2.0
        C = 1.0 + p(xi) * h / 2.0
        D = f(xi) * h * h
```

Листинг 2 — lab2.py. (2 часть)

```
        if i == 1:
            alpha[i] = -C / B
            beta[i] = (D - A * y_left) / B
        else:
            denominator = A * alpha[i - 1] + B
            alpha[i] = -C / denominator
            beta[i] = (D - A * beta[i - 1]) / denominator
    y_num[0] = y_left
    y_num[n] = y_right
    y_num[n - 1] = alpha[n - 1] * y_num[n] + beta[n - 1]
    for i in range(n - 2, 0, -1):
        y_num[i] = alpha[i] * y_num[i + 1] + beta[i]
    y_exact = [exact_solution(x) for x in x_values]
    errors = [abs(y_exact[i] - y_num[i]) for i in range(n + 1)]

    return x_values, y_exact, y_num, errors

def shooting_method():
    a = 0.0
    b = 1.0
    n = 10
    h = (b - a) / n
    y_left = 1.0
    y_right = math.exp(1.0) + math.sin(1.0)
    x_values = [a + i * h for i in range(n + 1)]
    y0 = [0.0] * (n + 1)
    y1 = [0.0] * (n + 1)
    y0[0] = y_left
    y0[1] = y_left + h
    y1[0] = 0.0
    y1[1] = h
    for i in range(1, n):
        xi = x_values[i]
        denominator = 1.0 + p(xi) * h / 2.0
        y0[i + 1] = (
            f(xi) * h * h
            + (2.0 - q(xi) * h * h) * y0[i]
            - (1.0 - p(xi) * h / 2.0) * y0[i - 1]
        ) / denominator
        y1[i + 1] = (
            (2.0 - q(xi) * h * h) * y1[i]
            - (1.0 - p(xi) * h / 2.0) * y1[i - 1]
        ) / denominator
    c1 = (y_right - y0[n]) / y1[n]
    y_num = [y0[i] + c1 * y1[i] for i in range(n + 1)]
    y_exact = [exact_solution(x) for x in x_values]
    errors = [abs(y_exact[i] - y_num[i]) for i in range(n + 1)]

    return x_values, y_exact, y_num, errors
```

Листинг 3 — lab2.py. (3 часть)

```
def print_comparison_table(x_values, y_exact, y_num_1, y_num_2, errors_1, errors_2):
    print("Comparison of methods")
    print(
        f"{'x':<10}"
        f"{'y':<22}"
        f"{'y*_1':<22}"
        f"{'y*_2':<22}"
        f"{'|y_i - y*_i|_1':<22}"
        f"{'|y_i - y*_i|_2':<22}"
    )
    for i in range(len(x_values)):
        print(
            f"{x_values[i]:<10.1f}"
            f"{y_exact[i]:<22.15f}"
            f"{y_num_1[i]:<22.15f}"
            f"{y_num_2[i]:<22.15f}"
            f"{errors_1[i]:<22.15f}"
            f"{errors_2[i]:<22.15f}"
        )

def main():
    x1, y1, y_num_1, err_1 = running_method()
    x2, y2, y_num_2, err_2 = shooting_method()
    print_method_table("Running method (method 1)", x1, y1, y_num_1, err_1)
    print_method_table("Shooting method (method 2)", x2, y2, y_num_2, err_2)
    print_comparison_table(x1, y1, y_num_1, y_num_2, err_1, err_2)

if __name__ == "__main__":
    main()
```

3.1 Результаты

В качестве исходных данных задаются следующие значения:

- отрезок разбиения: $[a, b] = [0, 1]$;
- число разбиений отрезка: $n = 10$ (число узлов $n + 1 = 11$);
- коэффициент при первой производной: $p(x) = 1$;
- коэффициент при искомой функции: $q(x) = -2$;
- правая часть дифференциального уравнения: $f(x) = \cos(x) - 3\sin(x)$;
- начальное условие для функции: $y(0) = 1$;
- начальное условие для первой производной: $y'(0) = 2$.

3.2 Вывод программы:

```
Метод прогонки (1 метод)
x          y                    y*                   |y_i - y*_i|
0.0        1.000000000000000    1.000000000000000    0.000000000000000
0.1        1.205004334722476    1.205111108412008    0.000106773689532
0.2        1.420072088955231    1.420266169662505    0.000194080707274
0.3        1.645379014237343    1.645640662458489    0.000261648221147
0.4        1.881243039949921    1.881551462550652    0.000308422600731
0.5        2.128146809304331    2.128479413988116    0.000332604683785
0.6        2.386761273785544    2.387092939327287    0.000331665541743
0.7        2.657970394708168    2.658272739142530    0.000302344434362
0.8        2.942897019391991    2.943137649488287    0.000240630096296
0.9        3.242930020784433    3.243071746807069    0.000141726022636
1.0        3.559752813266941    3.559752813266941    0.000000000000000

||y - y*|| = max|y_i - y*_i| = 0.000332604683785
```

Рис. 4 — Вывод программы в консоль (метод прогонки)

```
Метод стрельбы (2 метод)
x          y                    y*                   |y_i - y*_i|
0.0        1.000000000000000    1.000000000000000    0.000000000000000
0.1        1.205004334722476    1.205111108412007    0.000106773689531
0.2        1.420072088955231    1.420266169662504    0.000194080707273
0.3        1.645379014237343    1.645640662458487    0.000261648221145
0.4        1.881243039949921    1.881551462550649    0.000308422600728
0.5        2.128146809304331    2.128479413988113    0.000332604683781
0.6        2.386761273785544    2.387092939327284    0.000331665541740
0.7        2.657970394708168    2.658272739142527    0.000302344434359
0.8        2.942897019391991    2.943137649488285    0.000240630096294
0.9        3.242930020784433    3.243071746807068    0.000141726022635
1.0        3.559752813266941    3.559752813266941    0.000000000000000

||y - y*|| = max|y_i - y*_i| = 0.000332604683781
```

Рис. 5 — Вывод программы в консоль (метод стрельбы)

```
Сравнение методов
x          y                    y*_1                 y*_2                 |y_i - y*_i|_1        |y_i - y*_i|_2
0.0        1.000000000000000    1.000000000000000    1.000000000000000    0.000000000000000    0.000000000000000
0.1        1.205004334722476    1.205111108412008    1.205111108412007    0.000106773689532    0.000106773689531
0.2        1.420072088955231    1.420266169662505    1.420266169662504    0.000194080707274    0.000194080707273
0.3        1.645379014237343    1.645640662458489    1.645640662458487    0.000261648221147    0.000261648221145
0.4        1.881243039949921    1.881551462550652    1.881551462550649    0.000308422600731    0.000308422600728
0.5        2.128146809304331    2.128479413988116    2.128479413988113    0.000332604683785    0.000332604683781
0.6        2.386761273785544    2.387092939327287    2.387092939327284    0.000331665541743    0.000331665541740
0.7        2.657970394708168    2.658272739142530    2.658272739142527    0.000302344434362    0.000302344434359
0.8        2.942897019391991    2.943137649488287    2.943137649488285    0.000240630096296    0.000240630096294
0.9        3.242930020784433    3.243071746807069    3.243071746807068    0.000141726022636    0.000141726022635
1.0        3.559752813266941    3.559752813266941    3.559752813266941    0.000000000000000    0.000000000000000
```

Рис. 6 — Вывод программы в консоль (сравнение методов)

4 Выводы

В ходе выполнения лабораторной работы были изучены и реализованы два численных метода решения краевой задачи для линейного дифференциального уравнения второго порядка, а именно метод прогонки и метод стрельбы. Для каждого метода было получено численное решение на заданной сетке, после чего оно было сопоставлено с аналитическим решением.

Результаты тестирования показали, что оба метода дают практически одинаковые значения и обеспечивают высокую точность решения. Наиболее точным оказался метод стрельбы, так как его максимальная погрешность получилась немного меньше. Наименее точным оказался метод прогонки, однако разница между ними очень мала. Об этом говорит сравнение значений погрешности в узлах сетки и итоговой максимальной ошибки. Высокая точность результатов обоих методов связана с тем, что оба метода основаны на корректной аппроксимации одной и той же краевой задачи и применялись на одинаковой сетке.

С точки зрения эффективности оба метода являются достаточно быстрыми, поскольку вычисления выполняются на небольшом числе узлов и не требуют сложных операций. При этом метод прогонки обычно считается более выгодным, так как он специально предназначен для трёхдиагональных систем и использует прямой и обратный проход без лишних вычислений.

При увеличении числа разбиений результат будет становиться точнее, так как шаг сетки уменьшится и разностные приближения лучше воспроизведут поведение точного решения. Однако вместе с ростом точности будет увеличиваться время выполнения и объём вычислений, так как потребуется обрабатывать большее число узлов.